

BASES DE DATOS II

Sesión N°8

LABORATORIO PRÁCTICO

Fragmentación Vertical, Horizontal y Mixta

Motor de Base de Datos: PostgreSQL

Caso: Sistema de Gestión de una Cadena de Tiendas Retail

Universidad Tecnológica del Perú — UTP

Objetivos del Laboratorio

Al finalizar este laboratorio, el estudiante será capaz de:

- Crear esquemas de bases de datos distribuidas simuladas en PostgreSQL usando schemas separados.
- Aplicar fragmentación horizontal para dividir datos por criterio de fila (región, estado, categoría).
- Aplicar fragmentación vertical para dividir una tabla por columnas según frecuencia de acceso.
- Combinar ambas técnicas para escenarios complejos del mundo real.
- Reconstruir la tabla original mediante vistas (JOIN y UNION) y entender cuándo usar cada estrategia.

Resumen Conceptual: ¿Cuándo usar cada tipo?

Tipo	¿Cuándo usarlo?	Ejemplo del laboratorio
Horizontal	Los datos se acceden por grupos de filas (región, fecha, sucursal). Cada nodo atiende un grupo.	Ventas por región: Norte, Centro, Sur
Vertical	No todas las columnas se consultan igual. Las columnas frías y calientes tienen diferente nodo.	Empleados: datos de identidad vs. datos de nómina
Mixta	El volumen de datos es alto Y las columnas tienen acceso diferenciado.	Clientes: divididos por región Y por tipo de datos

PREPARACIÓN DEL ENTORNO

Creación del entorno de trabajo

Ejecuta los siguientes comandos en PostgreSQL para crear la base de datos y los schemas que simulan los nodos distribuidos:

```
-- Crear base de datos del laboratorio
CREATE DATABASE lab_fragmentacion;

-- Conectarse a la base de datos
\c lab_fragmentacion

-- Crear schemas que simulan nodos de la red distribuida
CREATE SCHEMA nodo_norte;
CREATE SCHEMA nodo_centro;
CREATE SCHEMA nodo_sur;
CREATE SCHEMA nodo_nomina;
CREATE SCHEMA nodo_operaciones;
CREATE SCHEMA nodo_central;

-- Verificar schemas creados
SELECT schema_name FROM information_schema.schemata
WHERE schema_name LIKE 'nodo_%';
```

EJERCICIO 1 — FRAGMENTACIÓN HORIZONTAL

Escenario: Ventas por Región Geográfica

Una cadena de tiendas retail tiene cientos de miles de ventas distribuidas en tres regiones del país. Los equipos de análisis en cada región solo necesitan consultar sus propias ventas. Aplicaremos FRAGMENTACIÓN HORIZONTAL dividiendo las filas por región.

Paso 1 — Crear la tabla base original (nodo central)

```
-- Tabla VENTAS en el nodo central (tabla maestra de referencia)
CREATE TABLE nodo_central.ventas (
  id_venta      SERIAL PRIMARY KEY,
  fecha         DATE NOT NULL,
  region        VARCHAR(20) NOT NULL,    -- 'norte', 'centro', 'sur'
  id_producto   INT NOT NULL,
  cantidad      INT NOT NULL,
  precio_unit   NUMERIC(10,2) NOT NULL,
  total         NUMERIC(12,2) GENERATED ALWAYS AS (cantidad * precio_unit) STORED
);
```

Paso 2 — Crear los fragmentos horizontales (un schema por región)

Cada fragmento tiene EXACTAMENTE el mismo esquema de columnas, pero diferente subconjunto de filas:

```
-- Fragmento Norte: solo ventas de la región norte
CREATE TABLE nodo_norte.ventas_norte (
  id_venta      INT PRIMARY KEY,
  fecha         DATE NOT NULL,
  region        VARCHAR(20) NOT NULL DEFAULT 'norte',
  id_producto   INT NOT NULL,
  cantidad      INT NOT NULL,
  precio_unit   NUMERIC(10,2) NOT NULL,
  total         NUMERIC(12,2) GENERATED ALWAYS AS (cantidad * precio_unit) STORED
);

-- Fragmento Centro: solo ventas de la región centro
CREATE TABLE nodo_centro.ventas_centro (
  id_venta      INT PRIMARY KEY,
  fecha         DATE NOT NULL,
  region        VARCHAR(20) NOT NULL DEFAULT 'centro',
  id_producto   INT NOT NULL,
  cantidad      INT NOT NULL,
  precio_unit   NUMERIC(10,2) NOT NULL,
  total         NUMERIC(12,2) GENERATED ALWAYS AS (cantidad * precio_unit) STORED
);

-- Fragmento Sur: solo ventas de la región sur
CREATE TABLE nodo_sur.ventas_sur (
  id_venta      INT PRIMARY KEY,
  fecha         DATE NOT NULL,
  region        VARCHAR(20) NOT NULL DEFAULT 'sur',
  id_producto   INT NOT NULL,
```

```

    cantidad      INT NOT NULL,
    precio_unit    NUMERIC(10,2) NOT NULL,
    total          NUMERIC(12,2) GENERATED ALWAYS AS (cantidad * precio_unit) STORED
);

```

Paso 3 — Insertar datos en los fragmentos

```

-- Datos región Norte (Lima, Trujillo, Piura)
INSERT INTO nodo_norte.ventas_norte (id_venta, fecha, id_producto, cantidad,
precio_unit)
VALUES
    (1, '2024-01-05', 101, 3, 49.90),
    (2, '2024-01-12', 205, 1, 299.00),
    (3, '2024-02-08', 101, 5, 49.90),
    (4, '2024-02-20', 310, 2, 120.00),
    (5, '2024-03-15', 205, 1, 299.00);

-- Datos región Centro (Huancayo, Ica, Ayacucho)
INSERT INTO nodo_centro.ventas_centro (id_venta, fecha, id_producto, cantidad,
precio_unit)
VALUES
    (6, '2024-01-07', 101, 2, 49.90),
    (7, '2024-01-18', 410, 4, 80.00),
    (8, '2024-02-14', 205, 1, 299.00),
    (9, '2024-03-03', 310, 3, 120.00),
    (10, '2024-03-22', 410, 6, 80.00);

-- Datos región Sur (Arequipa, Cusco, Puno)
INSERT INTO nodo_sur.ventas_sur (id_venta, fecha, id_producto, cantidad, precio_unit)
VALUES
    (11, '2024-01-10', 205, 2, 299.00),
    (12, '2024-02-02', 101, 8, 49.90),
    (13, '2024-02-28', 410, 3, 80.00),
    (14, '2024-03-11', 310, 1, 120.00),
    (15, '2024-03-30', 205, 1, 299.00);

```

Paso 4 — Reconstruir la tabla completa con UNION ALL (vista global)

La reconstrucción usa UNION ALL porque los fragmentos son DISJUNTOS (sin filas repetidas):

```

-- Vista que reconstruye toda la tabla de ventas
CREATE VIEW nodo_central.v_ventas_global AS
    SELECT * FROM nodo_norte.ventas_norte
    UNION ALL
    SELECT * FROM nodo_centro.ventas_centro
    UNION ALL
    SELECT * FROM nodo_sur.ventas_sur;

-- Probar la vista global
SELECT * FROM nodo_central.v_ventas_global ORDER BY id_venta;

```

Paso 5 — Consultas de análisis y verificación

```

-- 1. Total de ventas por región
SELECT region, COUNT(*) AS num_ventas, SUM(total) AS ingreso_total

```

```

FROM nodo_central.v_ventas_global
GROUP BY region ORDER BY ingreso_total DESC;

-- 2. Consulta LOCAL en el fragmento Norte (sin acceder al Sur ni Centro)
-- Esto es la VENTAJA de la fragmentación horizontal
SELECT fecha, id_producto, total
FROM nodo_norte.ventas_norte
WHERE fecha BETWEEN '2024-01-01' AND '2024-03-31'
ORDER BY total DESC;

-- 3. Producto más vendido en la región Sur
SELECT id_producto, SUM(cantidad) AS unidades
FROM nodo_sur.ventas_sur
GROUP BY id_producto ORDER BY unidades DESC LIMIT 1;

-- 4. Verificar que ningún id_venta se repite entre fragmentos
-- (condición de completitud y disjunción)
SELECT id_venta, COUNT(*) FROM (
    SELECT id_venta FROM nodo_norte.ventas_norte
    UNION ALL
    SELECT id_venta FROM nodo_centro.ventas_centro
    UNION ALL
    SELECT id_venta FROM nodo_sur.ventas_sur
) t GROUP BY id_venta HAVING COUNT(*) > 1;
-- Resultado esperado: 0 filas (no hay duplicados)

```

☒ Condiciones de correcta Fragmentación Horizontal

1. COMPLETITUD: Cada fila de la tabla original debe estar en algún fragmento. 2. DISJUNCIÓN: Ninguna fila aparece en más de un fragmento. 3. RECONSTRUCCIÓN: UNION ALL de todos los fragmentos devuelve la tabla original.

Preguntas de reflexión — Ejercicio 1

1. ¿Por qué usamos UNION ALL y no UNION en la vista global? ¿Qué diferencia hay?
2. Si una empresa abre tiendas en Iquitos (región Selva), ¿qué pasos exactos seguirías para agregar un cuarto fragmento?
3. ¿Qué pasa si se ejecuta la consulta completa de ventas globales desde el nodo Norte? ¿Genera tráfico de red? ¿Es eficiente?
4. Propón un criterio alternativo de fragmentación distinto a la región (por ejemplo, por trimestre o por rango de precio).

EJERCICIO 2 — FRAGMENTACIÓN VERTICAL

Escenario: Tabla de Empleados con Acceso Diferenciado por Departamento

La empresa tiene una tabla de empleados con información personal, operativa y de nómina. El área de Operaciones consulta nombre, cargo y horario constantemente. El área de RRHH consulta salario y beneficios. El área de IT gestiona los datos de acceso. Aplicaremos FRAGMENTACIÓN VERTICAL separando las columnas por perfil de acceso.

Tabla original completa (antes de fragmentar)

id_emp	nombre	cargo	turno	salario	beneficios	usuario_ad
1	Ana Flores	Supervisora	Mañana	4200.00	Seguro EPS	aflores
2	Luis Paredes	Vendedor	Tarde	2100.00	Essalud	lparedes
3	Rosa Quispe	Almacenera	Mañana	2000.00	Essalud	rquispe

Paso 1 — Crear los fragmentos verticales

```
-- Fragmento Operaciones: columnas de uso diario en tienda
-- Nodo: nodo_operaciones (acceso frecuente - consultado 100+ veces/día)
CREATE TABLE nodo_operaciones.emp_operaciones (
  id_emp      INT PRIMARY KEY,
  nombre      VARCHAR(100) NOT NULL,
  cargo       VARCHAR(60)  NOT NULL,
  turno       VARCHAR(20)  NOT NULL  -- 'Mañana', 'Tarde', 'Noche'
);

-- Fragmento Nómina: columnas sensibles de RRHH
-- Nodo: nodo_nomina (acceso restringido - solo fin de mes)
CREATE TABLE nodo_nomina.emp_nomina (
  id_emp      INT PRIMARY KEY,
  salario     NUMERIC(10,2) NOT NULL,
  beneficios  VARCHAR(60)
);

-- Fragmento IT: control de acceso a sistemas
-- Nodo: nodo_central (acceso técnico controlado)
CREATE TABLE nodo_central.emp_acceso (
  id_emp      INT PRIMARY KEY,
  usuario_ad  VARCHAR(30) NOT NULL  -- usuario Active Directory
);
```

Paso 2 — Insertar datos en cada fragmento vertical

```
-- Datos de operaciones
INSERT INTO nodo_operaciones.emp_operaciones VALUES
  (1, 'Ana Flores', 'Supervisora', 'Mañana'),
  (2, 'Luis Paredes', 'Vendedor', 'Tarde'),
  (3, 'Rosa Quispe', 'Almacenera', 'Mañana'),
```

```

    (4, 'Carlos Díaz', 'Cajero', 'Noche'),
    (5, 'Marta León', 'Vendedora', 'Tarde');

-- Datos de nómina (información sensible)
INSERT INTO nodo_nomina.emp_nomina VALUES
    (1, 4200.00, 'Seguro EPS'),
    (2, 2100.00, 'Essalud'),
    (3, 2000.00, 'Essalud'),
    (4, 1950.00, 'Essalud'),
    (5, 2050.00, 'Essalud');

-- Datos de acceso IT
INSERT INTO nodo_central.emp_acceso VALUES
    (1, 'aflores'),
    (2, 'lparedes'),
    (3, 'rquispe'),
    (4, 'cdiaz'),
    (5, 'mleon');

```

Paso 3 — Reconstruir la tabla original con JOIN (vista completa)

La reconstrucción usa JOIN porque los fragmentos comparten la clave primaria id_emp:

```

-- Vista que reconstruye la tabla completa de empleados
CREATE VIEW nodo_central.v_empleados_completo AS
SELECT
    o.id_emp,
    o.nombre,
    o.cargo,
    o.turno,
    n.salario,
    n.beneficios,
    a.usuario_ad
FROM nodo_operaciones.emp_operaciones o
JOIN nodo_nomina.emp_nomina n ON o.id_emp = n.id_emp
JOIN nodo_central.emp_acceso a ON o.id_emp = a.id_emp;

-- Ver la tabla completa reconstruida
SELECT * FROM nodo_central.v_empleados_completo;

```

Paso 4 — Consultas por perfil de acceso

```

-- ACCESO OPERACIONES: Cuadro de turnos (no toca datos de nómina)
SELECT nombre, cargo, turno
FROM nodo_operaciones.emp_operaciones
ORDER BY turno, nombre;

-- ACCESO RRHH: Planilla mensual (no toca datos de turno)
SELECT e.nombre, n.salario, n.beneficios,
    ROUND(n.salario * 12, 2) AS salario_anual
FROM nodo_nomina.emp_nomina n
JOIN nodo_operaciones.emp_operaciones e ON n.id_emp = e.id_emp
ORDER BY n.salario DESC;

-- ACCESO IT: Reporte de usuarios del sistema
SELECT e.nombre, a.usuario_ad
FROM nodo_central.emp_acceso a

```



```
JOIN nodo_operaciones.emp_operaciones e ON a.id_emp = e.id_emp;

-- ANÁLISIS: Promedio salarial por turno (requiere JOIN entre fragmentos)
SELECT o.turno, ROUND(AVG(n.salario), 2) AS salario_promedio
FROM nodo_operaciones.emp_operaciones o
JOIN nodo_nomina.emp_nomina n ON o.id_emp = n.id_emp
GROUP BY o.turno ORDER BY salario_promedio DESC;
```

Condiciones de correcta Fragmentación Vertical

1. COMPLETITUD: Todo atributo de la tabla original aparece en algún fragmento.
2. La CLAVE PRIMARIA (id_emp) debe estar en TODOS los fragmentos para permitir la reconstrucción.
3. RECONSTRUCCIÓN: JOIN de todos los fragmentos por la clave primaria devuelve la tabla original.

Diferencia fundamental con la fragmentación horizontal

Horizontal -> divide FILAS -> reconstrucción con UNION ALL Vertical -> divide COLUMNAS -> reconstrucción con JOIN La clave primaria DEBE repetirse en todos los fragmentos verticales (no es redundancia, es necesidad).

Preguntas de reflexión — Ejercicio 2

5. ¿Por qué en la fragmentación vertical el id_emp debe estar en TODOS los fragmentos y no solo en uno?
6. ¿Qué fragmento vertical conviene alojar en el servidor con más recursos de CPU? Justifica tu respuesta.
7. Si se agrega una columna 'fecha_ingreso', ¿a qué fragmento la asignarías? ¿Por qué?
8. ¿En qué situación una consulta de fragmentación vertical puede ser MÁS LENTA que consultar la tabla completa?

EJERCICIO 3 — FRAGMENTACIÓN MIXTA (Vertical + Horizontal)

Escenario: Base de Clientes de una Tienda Online con Distribución Nacional

La empresa tiene millones de clientes distribuidos en tres regiones. Además, se accede muy diferente a los datos de contacto (alta frecuencia) vs. datos financieros (solo para facturación). Necesitamos FRAGMENTACIÓN MIXTA: primero vertical (separar columnas por uso), luego horizontal (separar filas por región) en el fragmento de mayor acceso.

Diseño de la estrategia de fragmentación mixta

Fragmento	Columnas	División de filas
Contacto Norte	id, nombre, email, telefono, region	Solo clientes región 'norte'
Contacto Centro	id, nombre, email, telefono, region	Solo clientes región 'centro'
Contacto Sur	id, nombre, email, telefono, region	Solo clientes región 'sur'
Financiero	id, ruc_dni, limite_credito, estado_cuenta	Todos los clientes (un solo nodo)

Paso 1 — Crear los fragmentos mixtos

```
-- ===== FRAGMENTO VERTICAL 1: Datos de contacto =====
-- Luego subdividido horizontalmente por región

-- Contacto - Región Norte
CREATE TABLE nodo_norte.clientes_contacto_norte (
  id_cliente INT PRIMARY KEY,
  nombre     VARCHAR(120) NOT NULL,
  email      VARCHAR(100),
  telefono   VARCHAR(15),
  region     VARCHAR(20) DEFAULT 'norte'
);

-- Contacto - Región Centro
CREATE TABLE nodo_centro.clientes_contacto_centro (
  id_cliente INT PRIMARY KEY,
  nombre     VARCHAR(120) NOT NULL,
  email      VARCHAR(100),
  telefono   VARCHAR(15),
  region     VARCHAR(20) DEFAULT 'centro'
);

-- Contacto - Región Sur
CREATE TABLE nodo_sur.clientes_contacto_sur (
  id_cliente INT PRIMARY KEY,
  nombre     VARCHAR(120) NOT NULL,
  email      VARCHAR(100),
  telefono   VARCHAR(15),
  region     VARCHAR(20) DEFAULT 'sur'
);

-- ===== FRAGMENTO VERTICAL 2: Datos financieros =====
```

```
-- Sin fragmentación horizontal (acceso centralizado de facturación)
CREATE TABLE nodo_nomina.clientes_financiero (
  id_cliente      INT PRIMARY KEY,
  ruc_dni          VARCHAR(20),
  limite_credito   NUMERIC(12,2) DEFAULT 0,
  estado_cuenta    VARCHAR(20) DEFAULT 'activo'
);
```

Paso 2 — Insertar datos de prueba

```
-- Clientes Norte
INSERT INTO nodo_norte.clientes_contacto_norte VALUES
  (1, 'Pedro Ramírez', 'pedro.r@mail.com', '987001001', 'norte'),
  (2, 'Sofía Vargas', 'sofia.v@mail.com', '987001002', 'norte'),
  (3, 'Juan Huanca', 'juan.h@mail.com', '987001003', 'norte');

-- Clientes Centro
INSERT INTO nodo_centro.clientes_contacto_centro VALUES
  (4, 'Carmen Rojas', 'carmen.r@mail.com', '987002001', 'centro'),
  (5, 'Miguel Torres', 'miguel.t@mail.com', '987002002', 'centro'),
  (6, 'Lucía Soto', 'lucia.s@mail.com', '987002003', 'centro');

-- Clientes Sur
INSERT INTO nodo_sur.clientes_contacto_sur VALUES
  (7, 'Andrés Mamani', 'andres.m@mail.com', '987003001', 'sur'),
  (8, 'Elena Ccasa', 'elena.c@mail.com', '987003002', 'sur'),
  (9, 'Roberto Apaza', 'roberto.a@mail.com', '987003003', 'sur');

-- Datos financieros (todos en un nodo)
INSERT INTO nodo_nomina.clientes_financiero VALUES
  (1, '10234567', 5000.00, 'activo'),
  (2, '10345678', 2000.00, 'activo'),
  (3, '10456789', 3000.00, 'inactivo'),
  (4, '10567890', 4500.00, 'activo'),
  (5, '10678901', 1500.00, 'activo'),
  (6, '10789012', 6000.00, 'activo'),
  (7, '10890123', 2500.00, 'activo'),
  (8, '10901234', 3500.00, 'inactivo'),
  (9, '10012345', 4000.00, 'activo');
```

Paso 3 — Vistas de reconstrucción por niveles

```
-- NIVEL 1: Reconstruir todos los contactos (UNION ALL de fragmentos horizontales)
CREATE VIEW nodo_central.v_clientes_contacto AS
  SELECT * FROM nodo_norte.clientes_contacto_norte
  UNION ALL
  SELECT * FROM nodo_centro.clientes_contacto_centro
  UNION ALL
  SELECT * FROM nodo_sur.clientes_contacto_sur;

-- NIVEL 2: Reconstruir tabla completa (JOIN entre fragmento vertical contacto y financiero)
CREATE VIEW nodo_central.v_clientes_completo AS
SELECT
  c.id_cliente,
  c.nombre,
  c.email,
  c.telefono,
  c.region,
```

```

    f.ruc_dni,
    f.limite_credito,
    f.estado_cuenta
FROM nodo_central.v_clientes_contacto c
JOIN nodo_nomina.clientes_financiero f ON c.id_cliente = f.id_cliente;

-- Verificar reconstrucción completa
SELECT * FROM nodo_central.v_clientes_completo ORDER BY id_cliente;

```

Paso 4 — Consultas que demuestran la ventaja de la fragmentación mixta

```

-- CASO 1: Campaña de email solo para región Norte (acceso local, sin JOIN)
--          Ventaja: no toca nodo_centro, nodo_sur ni nodo_nomina
SELECT nombre, email
FROM nodo_norte.clientes_contacto_norte
WHERE email IS NOT NULL;

-- CASO 2: Revisar crédito de un cliente del Sur (JOIN puntual)
--          Solo se accede a un fragmento de contacto + fragmento financiero
SELECT c.nombre, f.limite_credito, f.estado_cuenta
FROM nodo_sur.clientes_contacto_sur c
JOIN nodo_nomina.clientes_financiero f ON c.id_cliente = f.id_cliente
WHERE c.nombre ILIKE '%Mamani%';

-- CASO 3: Reporte financiero global (solo toca nodo_nomina + contactos para el
nombre)
SELECT c.nombre, c.region, f.limite_credito
FROM nodo_central.v_clientes_contacto c
JOIN nodo_nomina.clientes_financiero f ON c.id_cliente = f.id_cliente
WHERE f.estado_cuenta = 'activo'
ORDER BY f.limite_credito DESC;

-- CASO 4: Cuántos clientes activos por región
SELECT c.region, COUNT(*) AS clientes_activos,
       SUM(f.limite_credito) AS credito_total
FROM nodo_central.v_clientes_completo c
JOIN nodo_nomina.clientes_financiero f ON c.id_cliente = f.id_cliente
WHERE f.estado_cuenta = 'activo'
GROUP BY c.region ORDER BY credito_total DESC;

```

Orden de reconstrucción en fragmentación mixta

Primero deshacer la fragmentación HORIZONTAL con UNION ALL (reconstruir todas las filas del fragmento de contacto). Luego deshacer la fragmentación VERTICAL con JOIN (combinar el fragmento de contacto con el financiero). El orden importa: siempre UNION ALL primero, luego JOIN.

Preguntas de reflexión — Ejercicio 3

9. Dibuja un diagrama de la arquitectura de nodos mostrando qué tabla vive en cada servidor. ¿Cuántos nodos físicos tendría este diseño?
10. ¿Por qué el fragmento financiero NO se dividió horizontalmente por región como el fragmento de contacto?
11. Si una consulta necesita el email y el límite de crédito de todos los clientes del Sur, ¿cuántos nodos intervienen? Enuméralos.

12. Propón una mejora al diseño: ¿habría algún beneficio en dividir también horizontalmente el fragmento financiero?

EJERCICIO INTEGRADOR — DESAFÍO FINAL

Caso Empresarial: Nuevo Módulo de Pedidos

La empresa quiere agregar un módulo de PEDIDOS al sistema. Tu equipo debe diseñar e implementar la fragmentación óptima desde cero.

Tabla completa de pedidos (sin fragmentar):

id_pedido	id_cliente	fecha	region	id_producto	cantidad	estado	monto_total
1	3	2024-04-01	norte	205	2	entregado	598.00
2	7	2024-04-02	sur	101	4	pendiente	199.60
3	5	2024-04-03	centro	310	1	entregado	120.00

Instrucciones del desafío:

- 13. Analiza los patrones de acceso: el área de Logística consulta region, estado y cantidad; el área de Finanzas consulta id_cliente, monto_total y fecha.
- 14. Diseña la estrategia de fragmentación más eficiente justificando el tipo elegido (horizontal, vertical o mixta).
- 15. Implementa en PostgreSQL las tablas de los fragmentos usando los schemas ya creados.
- 16. Inserta al menos 10 pedidos distribuidos entre las regiones.
- 17. Crea la(s) vista(s) de reconstrucción necesarias.
- 18. Escribe 3 consultas que demuestren la ventaja de la fragmentación elegida.
- 19. Calcula cuántos nodos intervienen en cada consulta y compáralo con acceder a la tabla completa.



Criterios de evaluación del desafío

- Correcta justificación del tipo de fragmentación elegido (20%)
- Tablas creadas con la estructura adecuada (20%)
- Datos insertados correctamente y distribuidos (20%)
- Vistas de reconstrucción funcionales (20%)
- Consultas que demuestran eficiencia (20%)

Tabla Comparativa Final: Los Tres Tipos de Fragmentación

Característica	Horizontal	Vertical	Mixta
¿Qué se divide?	Filas (registros)	Columnas (atributos)	Filas Y columnas
Criterio base	Valor de un atributo (región, fecha, estado)	Frecuencia de acceso a columnas	Ambos criterios combinados
Clave en fragmento	Solo en el fragmento al que pertenece la fila	En TODOS los fragmentos (obligatorio)	Según la dimensión: igual que H y V
Reconstrucción	UNION ALL de fragmentos	JOIN por clave primaria	UNION ALL + JOIN (en ese orden)
Mejor cuando...	Cada nodo atiende un grupo de usuarios distintos	El ancho de fila es grande y no todas las columnas se usan juntas	Ambas condiciones aplican
Ejemplo real	Ventas por región	Empleados: operaciones vs. nómina	Clientes: región + tipo de dato